

Engenharia de Software – PUC Campinas

Algoritmos e Linguagens de Programação

Prof. Dr. Paulo Cesar de Alvarenga Lucci

Char e String

char

Char e String

- CHAR

- Representa um caractere individual:

- `char letra = 'A';` //no caractere usamos apóstrofes ('caractere')

- Principais funções (biblioteca `ctype.h`):

- `bool islower(char a)`: Retorna se o argumento representa uma letra minúscula.

- `bool isupper(char a)`: Retorna se o argumento representa uma letra maiúscula.

- `char tolower(char a)`: Retorna o argumento na forma minúscula.

- `char toupper(char a)`: Retorna o argumento na forma maiúscula.

- `bool isdigit(char a)`: Retorna se o argumento é um dígito.

Char e String

○ LEITURA DE CHAR EM C

○ Cuidado com o `\n`:

○ `scanf("%c", &myChar);` // Lê o caractere, mas o Enter (`\n`) fica no buffer.

○ Isso pode interferir em leituras seguintes.

○ Solução simples e eficaz:

○ `scanf(" %c", &myChar);`

○ `getchar();` // Limpa o `\n` do buffer.

○ Assim evitamos comportamentos inesperados e mantém a leitura limpa

Obs:

O amarelo que aparece no `%c` é para evidenciar a recomendação de se colocar um **espaço** antes do `%c`.

O espaço informa ao `scanf()` para ignorar **qualquer caractere em branco** antes de ler o próximo **caractere útil**.

Isso inclui:

- espaços ()
- tabulações (`\t`)
- quebras de linha (`\n`)

Char e String

○ ESCRITA DE CHAR EM C

- Escrever um caractere em C é simples e direto;
- Usa-se a função `printf()` com o especificador `%c` (não precisa do espaço antes do `%c`).
- Exemplo:
 - `char sexo = 'M';`
 - `printf("Sexo: %c\n", sexo);`
 - Também pode ser usada para imprimir símbolos, letras e até valores ASCII.
 - Sem preocupações com buffers ou formatações especiais, diferente da leitura.

Char e String

○ EXEMPLO

```
#include <stdio.h>
#include <ctype.h> // Necessário para as funções com char

int main() {
    char c;

    printf("Digite um caractere: ");
    scanf(" %c", &c); // O espaço antes do %c ignora espaços em branco anteriores

    printf("\nAnálise do caractere '%c':\n", c);

    // Verifica se é minúscula
    if (islower(c)) {
        printf("- É uma letra minúscula.\n");
    }

    // Verifica se é maiúscula
    if (isupper(c)) {
        printf("- É uma letra maiúscula.\n");
    }

    // Converte para maiúscula e minúscula (sem alterar o original)
    printf("- Em minúscula: %c\n", tolower(c));
    printf("- Em maiúscula: %c\n", toupper(c));

    // Verifica se é dígito
    if (isdigit(c)) {
        printf("- É um dígito numérico.\n");
    }

    // Caso não seja letra nem dígito
    if (!isalpha(c) && !isdigit(c)) {
        printf("- Não é uma letra nem um dígito.\n");
    }

    return 0;
}
```

```
main.c
1  #include <stdio.h>
2  #include <ctype.h> // Necessário para as funções com char
3
4  int main() {
5      char c;
6
7      printf("Digite um caractere: ");
8      scanf(" %c", &c); // O espaço antes do %c ignora espaços em branco anteriores
9
10     printf("\nAnálise do caractere '%c':\n", c);
11
12     // Verifica se é minúscula
13     if (islower(c)) {
14         printf("- É uma letra minúscula.\n");
15     }
16
17     // Verifica se é maiúscula
18     if (isupper(c)) {
19         printf("- É uma letra maiúscula.\n");
20     }
21
22     // Converte para maiúscula e minúscula (sem alterar o original)
23     printf("- Em minúscula: %c\n", tolower(c));
24     printf("- Em maiúscula: %c\n", toupper(c));
25
26     // Verifica se é dígito
27     if (isdigit(c)) {
28         printf("- É um dígito numérico.\n");
29     }
30
31     // Caso não seja letra nem dígito
32     if (!isalpha(c) && !isdigit(c)) {
33         printf("- Não é uma letra nem um dígito.\n");
34     }
35
36     return 0;
37 }
```

string

Char e String

○ STRINGS

- Ao contrário de outras linguagens, C não tem um tipo de dado `string` nativo.
 - Em C, `strings` são representadas como um `vetor de caracteres`, terminadas pelo caractere nulo (`'\0'`).
 - `Strings` em C são manipuladas através de funções da biblioteca padrão `string.h`.

○ DECLARAÇÃO E INICIALIZAÇÃO:

- `Strings` são declaradas como `vetor` de caracteres: `char minhaString[20];`
- Podem ser inicializadas diretamente com aspas duplas:
 - `char minhaString[] = "Olá";` ou `//aqui irá ajustar automaticamente tamanho 4 ('O', 'l', 'á', '\0')`
 - `char minhaString[6] = "Olá";` `//terá 6 posições: 4 utilizadas e 2 não utilizadas.`
 - O caractere nulo (`'\0'`) é automaticamente adicionado no final da `string`, mesmo que não seja explicitamente declarado.

Char e String

- ACESSO E MANIPULAÇÃO:

- Funções comuns:

- `int strlen(string s)`: Retorna o tamanho da string (excluindo o `'\0'`).
 - `void strcpy(string b, string a)`: Copia uma string para outra (`b = a`).
 - `int strcmp(string str1, string str2)`: Compara se duas strings são idênticas ou não:
 - Negativo se `str1 < str2`.
 - Zero se `str1` for idêntica à `str2`.
 - Positivo se `str1 > str2`.
 - `char[] strcat(destino, origem)`: Concatenar duas strings.

Char e String

- ACESSO E MANIPULAÇÃO:

- Acesso individual aos caracteres:

- `minhaString[0]` acessa o primeiro caractere.
 - `minhaString[1]` o segundo, e assim por diante.
 - Desta forma podemos percorrer os caracteres de uma string até chegarmos em `'\0'`.
 - O caractere nulo (`'\0'`) indica o fim da **string** quando usada em funções como `printf("%s", minhaString)` ou ao percorrer a **string** em um loop.

```
main.c
1  #include <stdio.h>
2
3  int main() {
4      char cidade[] = "Rio de Janeiro";
5      int i;
6
7      printf("Caracteres da string \"%s\":\n", cidade);
8
9      for (i = 0; cidade[i] != '\0'; i++)
10     {
11         printf("cidade[%d] = %c\n", i, cidade[i]);
12     }
13
14     return 0;
15 }
```

Caracteres da string "Rio de Janeiro":
cidade[0] = R
cidade[1] = i
cidade[2] = o
cidade[3] =
cidade[4] = d
cidade[5] = e
cidade[6] =
cidade[7] = J
cidade[8] = a
cidade[9] = n
cidade[10] = e
cidade[11] = i
cidade[12] = r
cidade[13] = o

Char e String

○ LEITURA DE STRING EM C

- Utilizaremos o scanf com o tipo “`%[^\n]`”

- `char cidade[20];`

- `scanf("%[^\n]", cidade);` // Novamente o espaço antes de %

○ CUIDADO!!!

- **O comando de atribuição ‘=’ entre **duas** strings não funciona em C:**

- `char nome1[20] = "Paulo Cesar";` //Na inicialização ok!!!!!!

- `char nome2[20] = nome1;` // **NÃO FUNCIONA!!!** Para isso temos que utilizar a função strcpy.

Char e String

○ EXEMPLO

```
#include <stdio.h>
#include <string.h>

int main() {
    char nome[50], sobrenome[50], nomeCompleto[100], copia[50];

    // Leitura do nome e sobrenome
    printf("Digite seu nome: ");
    scanf("%s", nome);

    printf("Digite seu sobrenome: ");
    scanf("%s", sobrenome);

    // strlen
    printf("\nTamanho do nome: %lu caracteres\n", strlen(nome));
    printf("Tamanho do sobrenome: %lu caracteres\n", strlen(sobrenome));

    // strcpy
    strcpy(copia, nome);
    printf("Cópia do nome: %s\n", copia);

    // strcmp
    int resultado = strcmp(nome, sobrenome);
    if (resultado == 0) {
        printf("Nome e sobrenome são idênticos.\n");
    } else if (resultado < 0) {
        printf("Nome vem antes do sobrenome na ordem alfabética.\n");
    } else printf("Nome vem depois do sobrenome na ordem alfabética.\n");

    // strcat
    strcpy(nomeCompleto, nome); // Copia o nome para nomeCompleto
    strcat(nomeCompleto, " "); // Adiciona espaço
    strcat(nomeCompleto, sobrenome); // Adiciona sobrenome

    printf("Nome completo: %s\n", nomeCompleto);

    return 0;
}
```

```
main.c
1  #include <stdio.h>
2  #include <string.h>
3
4  int main() {
5      char nome[50], sobrenome[50], nomeCompleto[100], copia[50];
6
7      // Leitura do nome e sobrenome
8      printf("Digite seu nome: ");
9      scanf("%s", nome);
10
11     printf("Digite seu sobrenome: ");
12     scanf("%s", sobrenome);
13
14     // strlen
15     printf("\nTamanho do nome: %lu caracteres\n", strlen(nome));
16     printf("Tamanho do sobrenome: %lu caracteres\n", strlen(sobrenome));
17
18     // strcpy
19     strcpy(copia, nome);
20     printf("Cópia do nome: %s\n", copia);
21
22     // strcmp
23     int resultado = strcmp(nome, sobrenome);
24     if (resultado == 0) {
25         printf("Nome e sobrenome são idênticos.\n");
26     } else if (resultado < 0) {
27         printf("Nome vem antes do sobrenome na ordem alfabética.\n");
28     } else printf("Nome vem depois do sobrenome na ordem alfabética.\n");
29
30     // strcat
31     strcpy(nomeCompleto, nome); // Copia o nome para nomeCompleto
32     strcat(nomeCompleto, " "); // Adiciona espaço
33     strcat(nomeCompleto, sobrenome); // Adiciona sobrenome
34
35     printf("Nome completo: %s\n", nomeCompleto);
36
37     return 0;
38 }
```

```
Digite seu nome: Paulo
Digite seu sobrenome: Lucci

Tamanho do nome: 5 caracteres
Tamanho do sobrenome: 5 caracteres
Cópia do nome: Paulo
Nome vem depois do sobrenome na ordem alfabética.
Nome completo: Paulo Lucci
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```